EECS 182 Deep Neural Networks
Fall 2026 Gireeja Ranade

Homework 2

Due date: Sep 16, 2026, at 10:59 PM.

1. Stochastic Gradient Descent (when it is possible to interpolate)

This is a problem about the convergence of SGD for least-squares problems when there is actually a solution that achieves zero loss.

For simplicity, suppose that the problem we are given is

$$X\mathbf{w} = \mathbf{y} \tag{1}$$

where $X = \begin{bmatrix} \mathbf{x}_1^\top \\ \mathbf{x}_2^\top \\ \vdots \\ \mathbf{x}_n^\top \end{bmatrix}$ is a wide matrix with $\mathbf{x}_i$ being d-dimensional vectors and $\mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}$ is an n-dimensional vector. Here, we assume that X has full row-rank (i.e. the $\mathbf{x}_i$ are linearly independent) and so (1) indeed has solutions. (Note that as $d > n$, there are infinitely many solutions.)

While we already know lots of ways of solving this problem, it is an illustrative toy example to make sure we understand why SGD works in such settings. (This material was covered in lecture but you really do need to understand it yourself so you can deal with variations you might encounter as well as internalize the kinds of manipulations required.) This problem has a Demo Notebook (**demo link**) associated with it to help you play around with things to get an even deeper set of intuitions.

In this problem, we will just initialize $\mathbf{w}_0 = \mathbf{0}$ for simplicity.

(a) Let's do some preliminaries. First, we want to change coordinates to notationally simplify our analysis of SGD.

Let $\mathbf{w}^*$ be the min-norm solution to (1).

Write out what $\mathbf{w}^*$ is explicitly with respect to X and $\mathbf{y}$ and then, change coordinates to $\mathbf{w}' = \mathbf{w} - \mathbf{w}^*$ to write the new equations as:

$$X\mathbf{w}' = \mathbf{0} \tag{2}$$

What is the new initial condition for $\mathbf{w}_0'$?

(b) Next, let's leverage SVD coordinates to further simplify the problem. **Show that there exists an orthogonal matrix V giving the change of variables $\mathbf{w}'' = V^\top \mathbf{w}'$ so that (2) looks like**

$$[\widetilde{X} \;\; \mathbf{0}_{n\times(d-n)}]\mathbf{w}'' = \mathbf{0} \tag{3}$$

and furthermore, **show that the initial condition for $\mathbf{w}_0'$ you computed in the previous part, when viewed as $\mathbf{w}_0''$ has all zeros in the final $(d-n)$ positions.**

Hint: use the full SVD $X = U\Sigma V^\top$ and change only the parameter coordinates; do not multiply the equations by $U^\top$.

(c) Let $\widetilde{w} \in \mathbb{R}^n$ be the first n coordinates of $\mathbf{w}''$, and let $\widetilde{x}_i^\top$ be row i of $\widetilde{X} \in \mathbb{R}^{n \times n}$. **Argue why what you have seen in the previous parts allows us to now focus on a square system of equations:**

$$\widetilde{X}\widetilde{w} = \mathbf{0} \tag{4}$$

and furthermore show that each of the n constituent equations (corresponding to rows) of (4) can be obtained by means of coordinate changes from the same indexed equation in (1).

(d) Let's now engage with SGD itself. Here, we will just use a minibatch length of 1 and batch sampling with replacement. This means that at every iteration of SGD, we roll a fair n-sided die and choose the single equation in (1) that corresponds to the row that came up on the die. Let I_t be the iid uniform random variable on $\{1, \ldots, n\}$ sampled independently of all previous choices before updating $\mathbf{w}_t$ to $\mathbf{w}_{t+1}$.

At the $t+1$-th iteration, we compute

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \nabla \mathcal{L}_{I_t}(\mathbf{w}_t) \tag{5}$$

where $\mathcal{L}_i(\mathbf{w}) = (y[i] - \mathbf{x}_i^\top \mathbf{w})^2$ is the squared loss on the i-th equation and η is the step-size (learning rate).

Show that an SGD step taken in (5) for the original optimization problem matches exactly to an SGD step taken for $\widetilde{w}$ for solving (4), and that in particular these steps look like:

$$\widetilde{w}_{t+1} = \widetilde{w}_t - 2\eta \widetilde{x}_{I_t} \widetilde{x}_{I_t}^\top \widetilde{w}_t \tag{6}$$

(e) At this point, we can focus entirely on the simplified square system (4) and the stochastic evolution of the iterations described by (6).

To show convergence to zero, we need to pick a suitable stochastic Lyapunov function $\mathcal{L}(\widetilde{w})$ that is bounded below by zero and will decrease in expectation at every time step. Here, a Lyapunov function is a nonnegative measure of how far the iterate is from the solution. In particular, we want to establish

$$E[\mathcal{L}(\widetilde{w}_{t+1}) | \widetilde{w}_t] \leq (1 - \rho)\mathcal{L}(\widetilde{w}_t) \tag{7}$$

with a $1 > \rho > 0$ so that this Lyapunov function tends to decrease exponentially to zero. We will have to have a suitably small step-size/learning-rate η for this to happen, of course.

Show that (7) indeed implies that for every $\epsilon > 0$ and $\delta > 0$, there exists a $T > 0$ for which

$$P(\mathcal{L}(\widetilde{w}_T) < \epsilon) \geq 1 - \delta. \tag{8}$$

Hint: first take expectations over $\widetilde{w}_t$ and iterate the bound. Then use Markov's inequality $P(Z \geq \epsilon) \leq E[Z]/\epsilon$ for $Z \geq 0$.

(f) One natural guess for a stochastic Lyapunov function is

$$\mathcal{L}(\widetilde{w}) = \widetilde{w}^\top \widetilde{X}^\top \widetilde{X} \widetilde{w}. \tag{9}$$

Argue why the candidate Lyapunov function $\mathcal{L}(\widetilde{w})$ in (9) is non-negative and is only equal to zero at $\widetilde{w} = \mathbf{0}$.

(g) To prove (7), split the change in the Lyapunov function as

$$\mathcal{L}(\widetilde{w}_{t+1}) = \mathcal{L}(\widetilde{w}_t) + A + B \tag{10}$$

where the term A is linear in the actual stochastic update $(\widetilde{w}_{t+1} - \widetilde{w}_t)$ and the term B is quadratic in that update.

Expand out $\mathcal{L}(\widetilde{w}_t + (\widetilde{w}_{t+1} - \widetilde{w}_t))$ to give explicit forms for A and B.

(h) We are counting on the term A in (10) to give us actual contraction in expectation since this looks like a gradient-descent step. **Show:**

$$E[A|\widetilde{w}_t] \leq -c_1 \eta \mathcal{L}(\widetilde{w}_t) \tag{11}$$

where the $c_1 > 0$ is a positive constant that depends on the problem.

Hint: use the update (6) and the smallest singular value of $\widetilde{X}$. Why is that singular value positive?

(i) We need to make sure that the "quadratic" term B in (10) cannot undo the progress made by A in expectation. **Show:**

$$E[B|\widetilde{w}_t] \leq c_2 \eta^2 \mathcal{L}(\widetilde{w}_t) \tag{12}$$

where $c_2 > 0$ is another positive constant that depends on the problem.

Hint: substitute (6). Bound the quadratic form using the largest singular value of $\widetilde{X}$, then bound each row norm by $\beta = \max_i \|\widetilde{x}_i\|_2$.

(j) Finally, we can put the pieces together to see that

$$E[\mathcal{L}(\widetilde{w}_{t+1})|\widetilde{w}_t] \leq (1 - c_1\eta + c_2\eta^2)\mathcal{L}(\widetilde{w}_t) \tag{13}$$

where $c_1 > 0$ and $c_2 > 0$ as well. **Show that there exists a sufficiently small positive η so that $0 < 1 - c_1\eta + c_2\eta^2 < 1$, giving the contraction required in (7).**

(k) We can apply this interpolation argument to ridge regression through *feature augmentation*. Here is the needed reminder: for any data matrix X and $\lambda > 0$, set

$$\check{X} = [X \ \sqrt{\lambda} I_n], \qquad \boldsymbol{\theta} = \begin{bmatrix} \mathbf{w} \\ \mathbf{u} \end{bmatrix}.$$

The matrix $\check{X}$ is wide and has full row rank. The first d coordinates of the minimum-norm solution to $\check{X}\boldsymbol{\theta} = \mathbf{y}$ give the ridge-regression solution: eliminating $\mathbf{u} = (\mathbf{y} - X\mathbf{w})/\sqrt{\lambda}$ from $\|\boldsymbol{\theta}\|_2^2$ gives $\|\mathbf{w}\|_2^2 + \|\mathbf{y} - X\mathbf{w}\|_2^2/\lambda$.

Open the **demo notebook** and run the imports and helper-function cells at the top, followed by Part (4). Parts (1)–(3) are optional. Part (4) compares direct SGD on ridge regression ("Original Ridge") with SGD on the augmented system ("Feature Augmented"), both with a constant step size. **In a few sentences, compare the convergence of the squared parameter error $\|\mathbf{w}_t - \mathbf{w}^*_{\text{ridge}}\|_2^2$ in the two plots. Which keeps decreasing, and which settles around a nonzero error level?** *Hint: approximately straight downward trends on these plots, which have a logarithmic vertical axis, indicate geometric decay. Focus on the trend, not on fluctuations at individual iterations. You may reduce Part (4)'s* `N_iteration` *to* 100,000 *to shorten the run. The notebook's averaged-loss convention uses $\lambda = n\alpha$.*

2. Accelerating Gradient Descent with Momentum

Let $X \in \mathbb{R}^{n \times d}$ have full column rank ($n \geq d$), and let $y \in \mathbb{R}^n$. Thus $X^T X$ is invertible and all d singular values of X are positive. Consider finding the minimizer of the following objective:

$$\mathcal{L}(w) = ||y - Xw||_2^2 \tag{14}$$

Part (a) provides the gradient descent (GD) convergence results we will use. In this problem, we add momentum and see how it affects convergence. The update rule is

$$\begin{aligned} w_{t+1} &= w_t - \eta z_{t+1} \\ z_{t+1} &= (1-\beta) z_t + \beta g_t, \end{aligned} \tag{15}$$

where $g_t = \nabla \mathcal{L}(w_t)$, $\eta > 0$ is the learning rate, and β is the weight on the current gradient in the moving average. For the momentum analysis, assume $0 < \beta < 1$. Setting $\beta = 1$ instead gives ordinary gradient descent. A system is *stable* here if its error converges to zero from every initial state; this holds when every eigenvalue of its update matrix has magnitude less than 1.

We will see how an appropriate choice of momentum and learning rate can accelerate convergence.

(a) Recall that the gradient descent update of (14) is

$$w_{t+1} = \Big(I - 2\eta(X^T X)\Big) w_t + 2\eta X^T y \tag{16}$$

and the minimizer is

$$w^* = (X^T X)^{-1} X^T y \tag{17}$$

The geometric convergence factor for the parameter error is

$$\text{rate} = \max_i |1 - 2\eta\sigma_i^2| \tag{18}$$

A factor below 1 gives convergence; a smaller factor means faster decay. Choosing the learning rate that minimizes (18) gives the optimal learning rate

$$\eta^* = \frac{1}{\sigma_{\text{min}}^2 + \sigma_{\text{max}}^2}, \tag{19}$$

where σ_{max} and σ_{min} are the maximum and minimum singular value of the matrix X. The corresponding optimal convergence rate is

$$\text{optimal rate} = \frac{(\sigma_{\text{max}}/\sigma_{\text{min}})^2 - 1}{(\sigma_{\text{max}}/\sigma_{\text{min}})^2 + 1} \tag{20}$$

Therefore, how fast ordinary gradient descent converges is determined by the ratio between the maximum singular value and the minimum singular value as above.

Now, let's consider using momentum to smooth the gradients before taking a step in (15).

$$\begin{aligned} w_{t+1} &= w_t - \eta z_{t+1} \\ z_{t+1} &= (1-\beta) z_t + \beta(2X^T X w_t - 2X^T y) \end{aligned} \tag{21}$$

Use the SVD $X = U\Sigma V^T$, where V is a $d \times d$ orthogonal matrix and Σ has the same shape as X. In particular, $V^T X^T X V = \Sigma^T \Sigma = \mathrm{diag}(\sigma_1^2, \ldots, \sigma_d^2)$. This lets us express the parameter error and averaged gradient in the singular directions:

$$\begin{aligned} x_t &= V^T (w_t - w^*) \\ a_t &= V^T z_t. \end{aligned} \tag{22}$$

Please rewrite (21) with the reparameterized variables, $x_t[i]$ and $a_t[i]$. ($x_t[i]$ and $a_t[i]$ are i-th components of x_t and a_t respectively.)

(b) Notice that the above 2×2 vector/matrix recurrence has no external input. We can derive the 2×2 system matrix R_i from above such that

$$\begin{bmatrix} a_{t+1}[i] \\ x_{t+1}[i] \end{bmatrix} = R_i \begin{bmatrix} a_t[i] \\ x_t[i] \end{bmatrix} \tag{23}$$

Derive R_i.

(c) A symbolic calculation gives the characteristic polynomial

$$\begin{aligned} f(\lambda) &= \det(R_i - \lambda I) \\ &= (1-\beta-\lambda)(1-2\eta\beta\sigma_i^2 - \lambda) + 2\eta\beta(1-\beta)\sigma_i^2 \\ &= \lambda^2 - (2 - \beta - 2\eta\beta\sigma_i^2)\lambda + 1 - \beta. \end{aligned} \tag{24}$$

Its discriminant is

$$\begin{aligned} D &= (2-\beta-2\eta\beta\sigma_i^2)^2 - 4(1-\beta) \\ &= \beta(4\beta\eta^2\sigma_i^4 + 4\beta\eta\sigma_i^2 + \beta - 8\eta\sigma_i^2), \end{aligned} \tag{25}$$

Using the characteristic polynomial and discriminant above, find the two eigenvalues $\lambda_{1,i}$ and $\lambda_{2,i}$ of R_i in terms of η, β, σ_i, and D. When are they distinct and real, repeated and real, or non-real complex conjugates? State the condition on D in each case.

(d) **For the case when the eigenvalues are repeated, which learning rates η give this case, and are the eigenvalues stable (strictly inside the unit circle)? What is the highest such learning rate as a function of β and σ_i?**

(e) **For the case when the eigenvalues are distinct and real, what range of η keeps both eigenvalues strictly inside the unit circle? Express your answer in terms of β and σ_i.**
Hint: Use $f(1)$, $f(-1)$, and $\lambda_{1,i}\lambda_{2,i} = 1 - \beta \in (0,1)$ to locate the two roots inside $(-1,1)$, then combine this with $D > 0$.

(f) **Given result: complex eigenvalues.** For $0 < \beta < 1$ and $\sigma_i > 0$, define

$$\eta_- = \frac{2 - \beta - 2\sqrt{1-\beta}}{2\beta\sigma_i^2}, \qquad \eta_+ = \frac{2 - \beta + 2\sqrt{1-\beta}}{2\beta\sigma_i^2}.$$

Solving $D < 0$ gives the complete range with non-real complex eigenvalues:

$$\eta_- < \eta < \eta_+.$$

In this range, the eigenvalues are complex conjugates and their product is $1 - \beta$. Therefore,

$$|\lambda_{1,i}|^2 = \lambda_{1,i}\lambda_{2,i} = 1 - \beta, \qquad |\lambda_{1,i}| = |\lambda_{2,i}| = \sqrt{1 - \beta} < 1.$$

Thus every learning rate in this open interval gives a stable system. There is no largest learning rate with non-real eigenvalues: η must stay strictly below η_+. At either endpoint the eigenvalues are repeated and real. At η_- they both equal $+\sqrt{1-\beta}$, and at η_+ they both equal $-\sqrt{1-\beta}$; these endpoints are also stable.

The geometric convergence factor of a mode is $r_i = \max(|\lambda_{1,i}|, |\lambda_{2,i}|)$. Since the product of its eigenvalues is $1 - \beta$, we always have $r_i \geq \sqrt{1-\beta}$. Repeated or complex-conjugate eigenvalues attain this smallest possible factor. The overall factor is $r = \max_i r_i$, since the slowest singular direction determines the long-run rate. This is a geometric factor, not an exact finite-step error: repeated roots can also introduce a factor of t multiplying r_i^t. These results are provided for use in part (g).

(g) Consider a problem with two singular values $\sigma_{\min} = 1$ and $\sigma_{\max} = 2$, and fix $\beta = \frac{3}{4}$. Thus $\sigma_{\min}^2 = 1$, $\sigma_{\max}^2 = 4$, and $\sqrt{1-\beta} = \frac{1}{2}$.

Find the full interval of learning rates η that gives the fastest geometric convergence for gradient descent with momentum, and give one convenient choice from that interval. What are the best geometric convergence factors for momentum and for ordinary gradient descent?

For a simple iteration comparison, use the geometric factor r^T, where r is the method's best convergence factor. **For each method, find the smallest integer T such that $r^T \leq \frac{1}{4}$.** Small integer powers suffice. This comparison uses the geometric factors; exact finite-step parameter errors can also depend on transient effects.

(h) The two questions below use the Fall 2026 momentum notebook. It uses the same singular values and momentum parameter as part (g): $\sigma_{\min} = 1$, $\sigma_{\max} = 2$, and $\beta = \frac{3}{4}$, with $\eta_{\text{GD}} = \frac{1}{5}$ and $\eta_{\text{momentum}} = \frac{1}{3}$. Run the notebook and use the plots to answer the following questions. You do not need to submit the `ipynb` file. These are also the two questions in *Coding: Watching Momentum Work*; submit your answers only once.

How does the singular value σ_i influence the gradients and the parameter updates along direction i?

(i) Question: Comparing gradient descent and gradient descent with momentum, **which one converges faster for this task? Why?**

3. Optimizers

Here f_t is the loss on the minibatch used at step t, and g_t is its gradient. The momentum parameter β_1 below weights the previous average; it corresponds to $1-\beta$ in the preceding momentum problem. In Adam, squares, square roots, and division act elementwise. Use a small constant $\epsilon > 0$ in the denominator to avoid division by zero; bias correction is omitted in this exercise.

Algorithm 1 SGD with Momentum

Given $\eta = 0.001, \beta_1 = 0.9$
Initialize:
  time step $t \leftarrow 0$
  parameter $\theta_{t=0} \in \mathbb{R}^n$, $m_0 \leftarrow 0$
Repeat
  $t \leftarrow t + 1$
  $g_t \leftarrow \nabla f_t(\theta_{t-1})$
  $m_t \leftarrow \beta_1 m_{t-1} + (1-\beta_1) g_t$
  $\theta_t \leftarrow \theta_{t-1} - \eta m_t$
Until the stopping condition is met

Algorithm 2 Adam Optimizer (without bias correction)

Given $\eta = 0.001, \beta_1 = 0.9, \beta_2 = 0.999$
Initialize time step $t \leftarrow 0$, parameter $\theta_{t=0} \in \mathbb{R}^n$, $m_{t=0} \leftarrow 0$, $v_{t=0} \leftarrow 0$
Repeat
  $t \leftarrow t + 1$
  $g_t \leftarrow \nabla f_t(\theta_{t-1})$
  $m_t \leftarrow$ ____**(A)**____
  $v_t \leftarrow$ ____**(B)**____
  $\theta_t \leftarrow \theta_{t-1} - \eta \cdot \frac{m_t}{\sqrt{v_t}+\epsilon}$
Until the stopping condition is met

(a) Complete part **(A)** and **(B)** in the pseudocode of Adam.

(b) For this part, use **plain SGD without momentum or adaptive rescaling**, with learning rate $\eta > 0$ and regularization strength $\lambda \geq 0$. Establish the relationship between

- **L2 regularization** adds a squared-weight penalty to the minibatch loss:

$$f_t^{reg}(\theta) = f_t(\theta) + \frac{\lambda}{2}||\theta||_2^2$$

- **Weight decay** multiplies the weights by $1-\gamma$ at each update:

$$\theta_{t+1} = (1-\gamma)\theta_t - \eta \nabla f_t(\theta_t)$$

  Setting $\gamma = 0$ recovers plain SGD.

Show that SGD with weight decay using the original loss $f_t(\theta)$ is equivalent to regular SGD on the L2-regularized loss $f_t^{reg}(\theta)$ when γ is chosen correctly, and **find such a γ in terms of λ and η.**

4. Regularization and Instance Noise

Say we have m labeled data points $(\mathbf{x}_i, y_i)_{i=1}^m$, where each $\mathbf{x}_i \in \mathbb{R}^n$ and each $y_i \in \mathbb{R}$. We perform data augmentation by adding some noise to each vector every time we use it in SGD. This means for all points i, we have a true input $\mathbf{x}_i$ and add noise $\mathbf{N}_i$ to get the effective random input seen by SGD:

$$\check{\mathbf{X}}_i = \mathbf{x}_i + \mathbf{N}_i$$

The i.i.d. random noise vectors $\mathbf{N}_i$ are distributed as $\mathbf{N}_i \sim \mathcal{N}(\mathbf{0}, \sigma^2 I_n)$, with $\sigma^2 \geq 0$. The clean data and labels are fixed, and fresh noise is drawn independently each time a datapoint is used. Let $X \in \mathbb{R}^{m \times n}$ be the clean data matrix with rows $\mathbf{x}_i^\top$.

Arrange the augmented inputs as the rows of $\check{X} \in \mathbb{R}^{m \times n}$, so row i is $(\mathbf{x}_i + \mathbf{N}_i)^\top$, and let $\mathbf{y} = (y_1, \ldots, y_m)^\top$.

One way of thinking about what SGD might do is to consider learning weights that minimize the *expected* least squares objective for the **noisy** data matrix:

$$\operatorname*{argmin}_{\mathbf{w}} \mathbb{E}[\|\check{X}\mathbf{w} - \mathbf{y}\|^2] \tag{26}$$

(a) **Show that this problem (26) is equivalent to a regularized least squares problem:**

$$\operatorname*{argmin}_{\mathbf{w}} \frac{1}{m}\|X\mathbf{w} - \mathbf{y}\|^2 + \lambda\|\mathbf{w}\|^2 \tag{27}$$

You will need to determine the value of λ.

Hint: write the squared norm of a vector as an inner product, expand, and apply linearity of expectation.

Now consider a simplified example where we only have a single scalar datapoint $x \in \mathbb{R}$ with $x \neq 0$ and its corresponding label $y \in \mathbb{R}$. We are going to analyze this in the context of gradient descent. At update $t = 0, 1, \ldots$, from w_t to w_{t+1}, we use a noisy datapoint $\check{X}_t = x + N_t$ which is generated by adding different random noise values $N_t \sim \mathcal{N}(0, \sigma^2)$ to our underlying data point x. The noise values for each iteration of gradient descent are i.i.d., so the fresh N_t is independent of w_t. The loss used for this update is $\mathcal{L}_t(w) = \frac{1}{2}(\check{X}_t w - y)^2$. We initialize our weight to be $w_0 = 0$ and use a constant learning rate $\eta > 0$. Expectations below are over the noise draws. We analyze the mean $\mathbb{E}[w_t]$; its convergence alone does not establish convergence of individual noisy runs.

(b) Let w_t be the weight learned after the t-th iteration of gradient descent with data augmentation. **Write the gradient descent recurrence relation between $\mathbb{E}[w_{t+1}]$ and $\mathbb{E}[w_t]$ in terms of x, σ^2, y, and learning rate η.**

(c) **For what positive learning rates η does the mean recurrence converge to its fixed point from any initialization?**

(d) Assuming η is in the range from part (c), **what would we expect $\mathbb{E}[w_t]$ to converge to as $t \to \infty$? How does this differ from the optimal value of w if there were no noise being used to augment the data?**

(HINT: You can also use this to help check your work for part (a).)

5. Coding: Watching Momentum Work

This notebook shows how different singular directions converge using the exact least-squares example from the momentum analysis problem: singular values 2 and 1, momentum parameter $\beta = \frac{3}{4}$, and learning rates $\eta_{\text{GD}} = \frac{1}{5}$ and $\eta_{\text{momentum}} = \frac{1}{3}$. Signed errors and gradients show how each singular direction behaves, while the loss and error plots compare the two methods over time.

Open the Fall 2026 `q_sgd_momentum_analysis.ipynb` and run all cells. The notebook includes the data and optimizer implementations. You do not need to submit the `.ipynb` file; answer the questions below in your write-up. These repeat parts (h) and (i) of *Accelerating Gradient Descent with Momentum*. Submit each answer only once; a reference to your earlier answers is sufficient here.

If you would like to see the eigenvalues move as the learning rate changes, there is a companion notebook and a short video using the same numerical values.

(a) **How does σ_i influence the gradients and the parameter updates along direction i?**

(b) **Comparing gradient descent with and without momentum, which converges faster on this task, and why?**

Contributors:

- Matteo Guarrera.
- Mert Cemri.
- Sizhe Chen.
- Anant Sahai.
- Gireeja Ranade.
- Yaodong Yu.
- Suhong Moon.
- Gabriel Goh.
- Peter Wang.
- Yuxi Liu.
- Kevin Li.
- Saagar Sanghavi.

